Image Convolution and Filtering

Image Convolution

Definition:

- Mathematical operation combining image and kernel
- Fundamental to image processing and CNNs
- Creates various effects: blur, sharpen, edges

Process:

1. Place kernel over image region
2. Multiply corresponding elements
3. Sum the products
4. Slide to next position

Mathematical Notation:

$$(I * K)(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} I(x - i, y - j) \cdot K(i, j)$$

Key Parameters:

- **Kernel size:** 3×3, 5×5, etc.
- **Stride:** Step size (usually 1)
- **Padding:** Border handling

Convolution Example

Computing One Output Pixel:

$$\begin{bmatrix} 1 & 2 & 3 \\ 6 & 7 & 8 \\ 11 & 12 & 13 \end{bmatrix} * \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \frac{1}{9}(1+2+3+6+7+8+11+12+13) = 7$$

Steps:

1. Element-wise multiplication
2. Sum all products
3. Result is single output pixel
4. Repeat for entire image

Border Handling:

- Zero padding (add zeros)
- Replicate edges
- Reflect/mirror
- Valid (no padding)

Mean Filter

Concept:

- Replaces pixel with **average** of neighbors
- All weights equal
- Simple, fast noise reduction
- Also called box or averaging filter

Properties:

- Linear filter
- Uniform blur in all directions
- Reduces noise *and* detail
- Computationally efficient

Kernel Examples:

3×3:

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

5×5:

$$K = \frac{1}{25}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Mean Filter: Visual Comparison

Original

Mean 3×3

Mean 5×5

Mean 9×9

Observations:

- Larger kernel $\rightarrow$ more blur
- Sharp edges (tree, mountains) become progressively softer
- Fine details lost with larger kernels
- Uniform smoothing effect

Mean Filter: OpenCV Implementation

```python
import cv2
import numpy as np

# Read image
img = cv2.imread('lake.jpg')

# Apply mean filter with different kernel sizes
mean_3x3 = cv2.blur(img, (3, 3))
mean_5x5 = cv2.blur(img, (5, 5))
mean_9x9 = cv2.blur(img, (9, 9))

# Alternative: using boxFilter
mean_alt = cv2.boxFilter(img, -1, (5, 5), normalize=True)

# Display or save
cv2.imwrite('mean_filtered.jpg', mean_5x5)
```

Gaussian Filter

Concept:

- Uses Gaussian (bell curve) function
- **Weighted** average - center matters more
- Better edge preservation
- Industry-standard filter

2D Gaussian Function:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Key Properties:

- Isotropic (rotationally symmetric)
- Controlled by σ (standard deviation)
- Separable into 1D operations
- Natural-looking blur

Effect of σ:

- Small σ (<1): Subtle smoothing
- Medium σ (1-3): Balanced blur
- Large σ (>3): Heavy blur

Gaussian Kernel Structure

Example: 5×5 **Kernel** ($\sigma = 1$)

$$\frac{1}{273}\begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & \mathbf{41} & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$

- Center weight: $41/273 \approx 15\%$
- Decreases with distance
- Symmetric
- Sum $= 1$ (normalized)

Comparison with Mean Filter:

	Mean	**Gaussian**
Center weight	$1/9$	$41/273$
Edge weight	$1/9$	$1/273$
Distribution	Uniform	Bell-shaped

Gaussian Filter: Visual Comparison

Original

Gaussian 3×3

Gaussian 5×5

Gaussian 9×9

- Increasing kernel size $\rightarrow$ increased blur
- Edges remain more defined than mean filter
- Natural, smooth transition
- Better preserves image structure

Gaussian Filter: OpenCV Implementation

```python
import cv2

# Read image
img = cv2.imread('lake.jpg')

# Apply Gaussian blur
# Syntax: GaussianBlur(image, kernel_size, sigmaX, sigmaY=0)
gaussian_3x3 = cv2.GaussianBlur(img, (3, 3), sigmaX=0)
gaussian_5x5 = cv2.GaussianBlur(img, (5, 5), sigmaX=1)
gaussian_9x9 = cv2.GaussianBlur(img, (9, 9), sigmaX=2)

# If sigmaX=0, OpenCV calculates it from kernel size
# sigma = 0.3 * ((ksize - 1) * 0.5 - 1) + 0.8

# Different sigma for x and y directions (anisotropic)
anisotropic = cv2.GaussianBlur(img, (9, 9), sigmaX=3, sigmaY=1)

# Save result
cv2.imwrite('gaussian_filtered.jpg', gaussian_5x5)
```

Choosing the Right Parameters

Kernel Size Selection:

```python
# Rule of thumb:
# kernel_size = 6*sigma + 1

def calc_kernel(sigma):
    return 2*int(3*sigma) + 1

# Examples:
# sigma=1 -> kernel=7
# sigma=2 -> kernel=13
# sigma=3 -> kernel=19
```

Guidelines:

- Start small, increase gradually
- Kernel must be odd
- Larger kernels = slower

Custom Kernel Creation

```python
import numpy as np
import cv2

def create_gaussian_kernel(size, sigma):
    """Create a 2D Gaussian kernel"""
    ax = np.arange(-size // 2 + 1, size // 2 + 1)
    xx, yy = np.meshgrid(ax, ax)

    kernel = np.exp(-(xx**2 + yy**2) / (2 * sigma**2))
    return kernel / np.sum(kernel)  # Normalize

# Create custom kernel
kernel = create_gaussian_kernel(5, 1.0)
print(kernel)

# Apply using filter2D
img = cv2.imread('lake.jpg')
filtered = cv2.filter2D(img, -1, kernel)
```